

# Hybrid Semantic Prototype Weighting and Learning-to-Rank for Automated Bug Assignment

Suyash Srivastava  
Independent Researcher  
India  
suyash1527@gmail.com.com

**Abstract**—Automated bug assignment aims to reduce the triage time in software maintenance by recommending the most suitable developers to fix newly reported bugs. Traditional approaches rely on static keyword-based heuristics or simple similarity metrics that fail to capture the nuanced semantic intent of a bug report. In this paper, we propose a novel pipeline consisting of Hybrid Semantic Prototype Weighting paired with a Learning-to-Rank (LTR) model. First, a Weighted Sum Model (WSM) computes candidate scores using dynamic weights derived from cosine similarities between the bug embedding and six curated "prototype sentences" representing ideal developer attributes. Second, an XGBoost Ranker refines the candidate order using historical resolution data and 14 extracted features, incorporating a confidence-based fallback mechanism. Evaluated on a real-world Eclipse dataset consisting of 10,000 bug reports and 209 unique developers, our LTR approach achieves a Top-1 accuracy of 31.50% and a Mean Reciprocal Rank (MRR) of 0.4717, outperforming the semantic WSM baseline by an absolute 9.50% in Top-1 accuracy.

**Index Terms**—Bug Assignment, Machine Learning, Natural Language Processing, Learning to Rank, Semantic Embeddings, XGBoost

## I. INTRODUCTION

As software systems grow in complexity, bug tracking repositories (e.g., Bugzilla, Jira) receive hundreds of bug reports daily. Manual triage is a time-consuming and error-prone process. The bug assignment problem involves recommending a ranked list of candidate developers who possess the right expertise, availability, and historical context to resolve a given bug efficiently.

While existing approaches often use Term Frequency-Inverse Document Frequency (TF-IDF) or topic modeling combined with classification algorithms (like SVM or Naive Bayes), they often struggle with vocabulary mismatch and synonyms. Furthermore, traditional heuristics rely on hardcoded keyword lists to assess aspects like "urgency" or "experience required."

In this work, we introduce a hybrid framework that bridges modern semantic Natural Language Processing (NLP) with Learning-to-Rank algorithms. We replace brittle keyword lists with Semantic Prototype Weighting, dynamically scoring bug reports against "ideal" natural language sentences using dense transformer embeddings. Finally, we train an XGBRanker to optimize the final developer rankings, ensuring high precision while maintaining robust fallback behaviors.

## II. METHODOLOGY

Our automated bug assignment pipeline consists of eight sequential steps, combining semantic search, multi-criteria decision making, and learning-to-rank.

### A. Step 1: Hybrid Feature Extraction

Given a new bug report  $B_{new}$ , we extract both dense and sparse representations of its textual description.

- **Dense Semantic Embedding ( $E_{dense}$ )**: Generated using a pre-trained SentenceTransformer (all-MiniLM-L6-v2), producing a 384-dimensional vector.
- **Sparse Lexical Embedding ( $E_{sparse}$ )**: Generated using a TF-IDF vectorizer (unigrams and bigrams,  $0.0 < df < 0.8$ ).

### B. Step 2: Hybrid Similarity Search

We compute the cosine similarity between  $B_{new}$  and all historical bugs  $H_i$ . The final similarity score  $S(B_{new}, H_i)$  is a balanced hybrid:

$$S(B_{new}, H_i) = 0.5 \cdot \cos(E_{dense}^{new}, E_{dense}^{H_i}) + 0.5 \cdot \cos(E_{sparse}^{new}, E_{sparse}^{H_i}) \quad (1)$$

### C. Step 3 & 4: Candidate Extraction and Hard Filtering

We select the top  $K = 20$  most similar historical bugs and extract the set of developers who successfully resolved them. This set forms the initial candidate pool  $C$ . We apply a hard filter to remove any developer  $d \in C$  who is currently on leave or whose current workload exceeds a maximum threshold ( $Workload \geq 0.95$ ).

### D. Step 5: KNN Affinity

For each available candidate  $d$ , we calculate a topical expertise score, termed KNN Affinity ( $A_d$ ). It is defined as the mean hybrid similarity between  $B_{new}$  and the subset of historical bugs  $H_d$  previously fixed by developer  $d$ :

$$A_d = \frac{1}{|H_d|} \sum_{h \in H_d} S(B_{new}, h) \quad (2)$$

### E. Step 6: Attribute Matrix Construction

For the filtered candidates, we construct a feature matrix  $X$  where each row corresponds to a candidate. The matrix consists of 6 core attributes:

- 1) **Experience** ( $Exp_d$ ): Historical total fixes.
- 2) **Fix Time** ( $FT_d$ ): Average resolution time.
- 3) **Success Rate** ( $SR_d$ ): Ratio of successful fixes to total assignments.
- 4) **Workload** ( $WL_d$ ): Current concurrent bug assignments.
- 5) **Domain Skill** ( $DS_d$ ): Expertise in the bug's specific module.
- 6) **KNN Affinity** ( $A_d$ ): As defined above.

### F. Step 7: Semantic Prototype Weighting

To dynamically compute the relative importance (weights) of the 6 attributes without hardcoded heuristics, we introduce Semantic Prototypes. We define 6 natural language sentences  $P_j$  ( $j \in \{1..6\}$ ), each describing the "ideal" bug for a specific attribute (e.g., the fix-time prototype describes an urgent, production-down scenario).

The raw prototype score  $R_j$  is the cosine similarity between the dense embedding of  $B_{new}$  (appended with its severity label) and the embedding of  $P_j$ :

$$R_j = \frac{\cos(E_{dense}^{new}, E_{dense}^{P_j}) + 1.0}{2.0} \quad (3)$$

We apply a multiplicative severity modifier  $M_{sev}$  to amplify relevant prototypes (e.g., Critical severity boosts Fix Time and Success Rate, while Minor severity boosts Workload). The final normalized weight vector  $W$  is calculated as:

$$W_j = \frac{(R_j \cdot M_{sev}^{(j)}) + 0.05}{\sum_{k=1}^6 ((R_k \cdot M_{sev}^{(k)}) + 0.05)} \quad (4)$$

### G. Step 8: Learning-to-Rank (LTR)

The Weighted Sum Model (WSM) baseline computes final developer scores by taking the dot product of the Min-Max normalized attribute matrix and the dynamic weight vector  $W$ , followed by a 1.1x multiplier for exact component matches.

To improve upon WSM, we treat assignment as a ranking problem. We extract a 14-dimensional feature vector for each candidate:

- **Category A (6 features):** The 6 core developer attributes.
- **Category B (4 features):** Bug-Developer interactions (Exact Component Match boolean, number of matched bugs in Top-K, maximum historical similarity, mean historical similarity).
- **Category C (3 features):** One-hot encoded bug severity (Critical, Normal, Minor).
- **Category D (6 features):** The 6 normalized Semantic Prototype scores.

An **XGBRanker** model is trained using the pairwise learning-to-rank objective (`rank:ndcg`) with 5-fold cross validation.

**Safety Guardrail (Confidence Fallback):** During inference, if the LTR model's confidence gap (the difference in

predicted score between the Top-1 and Top-2 candidates) is below a minimum threshold ( $\Delta < 0.1$ ), the system automatically falls back to the WSM scoring to guarantee stability.

## III. EXPERIMENTAL SETUP

### A. Dataset

The system was evaluated on a real-world dataset extracted from the Eclipse bug tracking repository.

- **Total Resolvable Bugs:** 10,000
- **Training Set:** 9,800 bugs (chronological split)
- **Test Set:** 200 bugs
- **Developer Pool:** 209 unique developers

### B. Evaluation Metrics

We report Top- $N$  Accuracy (the percentage of bugs where the actual resolver appeared in the Top  $N$  recommendations) for  $N \in \{1, 3, 5\}$ . We also report the Mean Reciprocal Rank (MRR), which averages the multiplicative inverse of the rank of the correct developer:

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i} \quad (5)$$

## IV. RESULTS AND DISCUSSION

The evaluation results on the 200-bug test set are summarized in Table I.

TABLE I  
PERFORMANCE COMPARISON: WSM VS LTR

| Metric           | Baseline (WSM) | XGBRanker (LTR) | Improvement |
|------------------|----------------|-----------------|-------------|
| Top-1 Accuracy   | 22.00%         | <b>31.50%</b>   | +9.50%      |
| Top-3 Accuracy   | 49.50%         | <b>60.00%</b>   | +10.50%     |
| Top-5 Accuracy   | 65.00%         | <b>70.50%</b>   | +5.50%      |
| MRR              | 0.3954         | <b>0.4717</b>   | +0.0763     |
| Latency (ms/bug) | 65.6 ms        | 152.5 ms        | -86.9 ms    |

The XGBRanker model significantly outperforms the Semantic Prototype WSM baseline. The Top-1 accuracy improved by an absolute 9.50%, bringing the total accuracy to 31.50%. Similarly, Top-3 accuracy increased from 49.50% to 60.00%.

This substantial improvement highlights the XGBRanker's ability to learn complex, non-linear interactions between developer workloads, historical similarities, and semantic prototype scores that a simple linear Weighted Sum Model cannot capture.

While the inference latency increased from 65.6 ms to 152.5 ms per bug due to the overhead of tree-based prediction and comprehensive candidate feature extraction, this latency remains well within acceptable bounds for real-world asynchronous bug triage systems (which typically operate with Service Level Agreements measured in seconds or minutes).

## V. CONCLUSION

This paper presented a modern architecture for automated bug assignment, eschewing rigid keyword heuristics in favor of dense semantic prototypes and pairwise learning-to-rank. By combining SentenceTransformers with an XGBoost Ranker, we achieved a highly robust triage pipeline. Experimental results on the Eclipse dataset demonstrated a significant 9.5% absolute improvement in Top-1 routing accuracy. Future work will explore continuous online learning to update the XGBRanker in real-time as developers join or leave the organization.